

Blocco 4 - Prime query SQL

Leggere una relazione con SELECT, WHERE e ORDER BY

Relational Databases & SQL

30 min teoria + 90 min pratica

1. Rappresentazione iniziale

Partiamo da una soluzione semplice e concreta, prima di introdurre il modello generale.

Perché questo blocco

Problema didattico

Dopo aver costruito uno schema, il primo problema è formulare domande precise: quali righe vogliamo vedere, quali colonne servono, quali valori vanno calcolati e in quale ordine leggere il risultato.

Idea guida

Prima osserviamo una rappresentazione naturale, poi ne discutiamo i limiti e solo dopo formalizziamo.

Leggere una tabella con SELECT *

Situazione concreta

La prima query spontanea è mostrare tutta una tabella. Funziona per orientarsi, ma non è ancora una domanda precisa.

```
SELECT * FROM products;
```

Limiti della rappresentazione iniziale

- ▶ restituisce colonne non necessarie;
- ▶ non dichiara quali righe interessano;
- ▶ non garantisce ordine;
- ▶ non calcola informazioni derivate;
- ▶ non gestisce NULL in modo intenzionale.

2. Soluzione più generale

Riorganizziamo l'informazione in una forma più flessibile e controllabile.

Da ispezione a domanda precisa

Principio

Una query didatticamente corretta indica relazione di partenza, righe da tenere, colonne da produrre e ordinamento del risultato.

Esempio di riorganizzazione

```
SELECT product_id, sku, unit_price  
FROM products  
WHERE active  
ORDER BY unit_price DESC, product_id;
```


Processo di trasformazione

- ❶ scegliere la tabella con la granularità corretta;
- ❷ filtrare le righe con `WHERE`;
- ❸ scegliere colonne e calcoli con `SELECT`;
- ❹ ordinare con `ORDER BY`;
- ❺ controllare i valori mancanti con `IS NULL`.

Analogia: richiesta a un archivista

Analogia o caso esplicativo

Dire “dammi i prodotti” è vago. Dire “dammi codice, nome e prezzo dei prodotti attivi, ordinati dal più caro al meno caro” è una richiesta verificabile. Una query SQL deve avere questa precisione.

3. Formalizzazione

Diamo un nome preciso ai concetti emersi dall'esempio.

Formalizzazione: query base

- ▶ FROM: relazione sorgente;
- ▶ WHERE: selezione di tuple;
- ▶ SELECT: proiezione e calcolo;
- ▶ ORDER BY: ordinamento del risultato;
- ▶ NULL: assenza di valore noto.

- ▶ FROM: relazione sorgente;
- ▶ WHERE: filtro sulle righe;
- ▶ SELECT: colonne mostrate o calcolate;
- ▶ ORDER BY: ordine di lettura del risultato;
- ▶ LIMIT: taglio finale del risultato.

Come riconoscere una buona soluzione

- ▶ la tabella scelta ha la granularità giusta;
- ▶ i filtri sono leggibili e verificabili;
- ▶ le colonne calcolate hanno alias chiari;
- ▶ il risultato ha un ordinamento esplicito quando serve.

4. Esempio guidato

Applichiamo il modello generale a un caso piccolo, leggendo ogni passaggio.

Esempio: situazione iniziale

Domanda o frase di dominio

Domanda: “Quali prodotti attivi sono più costosi, e in che fascia prezzo cadono?”

Esempio: ragionamento

- ▶ la granularità è una riga per prodotto;
- ▶ la tabella sorgente è `products`;
- ▶ il filtro è `active`;
- ▶ la fascia prezzo è una colonna derivata calcolata con `CASE`.

Esempio completo: query base

```
SELECT product_id, sku, product_name, unit_price,  
       CASE  
         WHEN unit_price < 100 THEN 'low'  
         WHEN unit_price < 700 THEN 'medium'  
         ELSE 'high'  
       END AS price_band  
FROM products  
WHERE active  
ORDER BY unit_price DESC, product_id;
```

Errori frequenti da prevenire

- ▶ scrivere `SELECT *` anche quando servono poche colonne;
- ▶ confondere filtro su righe e scelta delle colonne;
- ▶ usare `= NULL` invece di `IS NULL`;
- ▶ filtrare date con intervalli ambigui;
- ▶ ordinare senza gestire i pari merito.

5. Pratica guidata

Ripetiamo lo stesso processo su un problema diverso: rappresentazione iniziale, limiti, riorganizzazione, soluzione.

Esercitazione: problema informale

Contesto

Il responsabile commerciale vuole una prima esplorazione del database: prodotti attivi, clienti recenti, ordini aperti, importi netti e spedizioni non concluse.

Esercitazione: specifica corretta

- ▶ input: schema training caricato in PostgreSQL;
- ▶ output: una query per ciascuna domanda operativa;
- ▶ vincolo: ogni query deve dichiarare ordinamento e alias delle colonne calcolate;
- ▶ vincolo: i valori mancanti vanno gestiti con `IS NULL` o `IS NOT NULL`.

Esercitazione: definizione della soluzione

- ➊ partire sempre dalla relazione con la granularità richiesta;
- ➋ scrivere query brevi e verificabili una alla volta;
- ➌ usare CASE solo dopo aver controllato i valori sorgente;
- ➍ salvare le query finali in `solution.sql`.

Implementazione completa: query rappresentative (1/2)

```
SET search_path TO training;
```

```
SELECT product_id, sku, product_name, unit_price  
FROM products  
WHERE active  
ORDER BY unit_price DESC, product_id;
```

```
SELECT customer_id, full_name, country, signup_date  
FROM customers  
WHERE signup_date >= DATE '2025-02-01'  
ORDER BY signup_date, customer_id;
```


Implementazione completa: query rappresentative (2/2)

```
SELECT order_id, customer_id, order_date, status, channel
FROM orders
WHERE status IN ('pending', 'cancelled', 'refunded')
ORDER BY order_date, order_id;
```

```
SELECT order_id, product_id,
       round(quantity * unit_price * (1 - discount_pct / 100.0), 2) AS net_amount
FROM order_items
ORDER BY order_id, product_id;
```

```
SELECT shipment_id, order_id, shipped_at, delivered_at
FROM shipments
WHERE delivered_at IS NULL
ORDER BY shipped_at, shipment_id;
```

Esercitazione: organizzazione dei 90 minuti

- ▶ 10 min esplorazione tabelle e colonne;
- ▶ 25 min filtri semplici su prodotti, clienti e ordini;
- ▶ 25 min colonne derivate e CASE;
- ▶ 20 min gestione di NULL e date;
- ▶ 10 min confronto sulle soluzioni.

- ▶ confrontare almeno due soluzioni quando esistono alternative;
- ▶ chiedere quali assunzioni cambierebbero la soluzione;
- ▶ separare errori di sintassi, errori di modello ed errori di interpretazione.

- ▶ una query base è corretta se la domanda e la granularità sono chiare;
- ▶ WHERE decide quali righe entrano, SELECT decide cosa mostrare;
- ▶ un risultato senza ordinamento non va usato come riferimento stabile.